

UNIVERSIDADE DO VALE DO ITAJAÍ
CURSO DE CIÊNCIA DA COMPUTAÇÃO

[NOME COMPLETO DO ALUNO 1]

[NOME COMPLETO DO ALUNO 2]

[NOME COMPLETO DO ALUNO 3]

**SISTEMA DE PROCESSAMENTO PARALELO DE
REQUISIÇÕES A UM BANCO DE DADOS**

Comunicação entre processos, threads e exclusão mútua

Disciplina: Sistemas Operacionais

Avaliação M1

Professor: [nome do professor]

Itajaí

2026

1 INTRODUÇÃO

Sistemas operacionais modernos executam vários processos ao mesmo tempo, e esses processos frequentemente precisam trocar informações entre si. Como cada processo tem seu próprio espaço de endereçamento, essa troca não pode ser feita por variáveis comuns: é necessário algum mecanismo de comunicação entre processos, ou IPC (Inter-Process Communication).

Além disso, um mesmo processo pode ser dividido em várias linhas de execução, as threads, que compartilham a memória entre si. Isso permite atender a vários pedidos em paralelo, mas cria um problema novo: se duas threads alterarem a mesma estrutura ao mesmo tempo, os dados podem ficar inconsistentes. A solução é delimitar a seção crítica e garantir exclusão mútua.

Este trabalho implementa um sistema que reúne esses três conceitos. Um processo cliente envia requisições a um banco de dados simulado por meio de memória compartilhada; um processo servidor recebe essas requisições e as processa com um conjunto de quatro threads; e o acesso à tabela de dados, compartilhada por todas as threads, é protegido por um mutex.

O objetivo é demonstrar, em um programa que funciona de ponta a ponta, como os processos se comunicam sem compartilhar variáveis, como as threads são criadas e coordenadas, e qual é o efeito prático da exclusão mútua sobre o resultado e sobre o tempo de execução.

2 O PROBLEMA PROPOSTO

O enunciado da avaliação pede o desenvolvimento de um sistema que simule o funcionamento interno de um gerenciador de requisições a um banco de dados, com os seguintes componentes: um processo cliente, que envia requisições de consulta ou inserção por meio de IPC — pipe ou memória compartilhada; um processo servidor, que recebe as requisições e utiliza várias threads para processá-las em paralelo; e o uso de mutex ou semáforo para garantir acesso seguro a uma estrutura compartilhada que simula o banco de dados.

O sistema deve suportar quatro operações: INSERT, para adicionar um registro; DELETE, para remover um registro; SELECT, para buscar dados a partir do identificador; e UPDATE, para alterar um campo de um registro existente. A estrutura interna do banco é definida pelo próprio enunciado como um registro contendo um identificador inteiro e um nome de até cinquenta caracteres.

São requisitos técnicos obrigatórios que cliente e servidor sejam executáveis distintos, rodando como processos separados do sistema operacional; que a comunicação entre eles use IPC real, e não variáveis globais ou leitura repetida de arquivos temporários; que o servidor utilize múltiplas threads; e que o acesso à tabela compartilhada seja protegido por mutex ou semáforo. As respostas podem ser gravadas em um segundo canal de IPC ou em um arquivo de log.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Comunicação entre processos

Cada processo enxerga apenas o seu próprio espaço de endereçamento, o que impede que um leia diretamente as variáveis do outro. Para que dois processos troquem dados, o sistema operacional precisa oferecer um mecanismo próprio (TANENBAUM; BOS, 2016). A memória compartilhada é um desses mecanismos: o sistema mapeia a mesma região física de memória no espaço de endereçamento dos dois processos, e a partir daí ambos leem e escrevem nela como se fosse memória comum.

Por ser um recurso compartilhado, a memória precisa de coordenação. Se o cliente escreve enquanto o servidor lê a mesma posição, o resultado é imprevisível. Essa coordenação é feita com semáforos, que são contadores mantidos pelo sistema operacional sobre os quais só existem duas operações: uma que decrementa e bloqueia o processo quando o valor chega a zero, e outra que incrementa e libera quem estiver bloqueado (SILBERSCHATZ; GALVIN; GAGNE, 2015).

3.2 Threads e paralelismo

Uma thread é uma linha de execução dentro de um processo. Diferentemente de processos distintos, as threads de um mesmo processo compartilham o espaço de endereçamento, o que torna a troca de informação entre elas imediata e a criação de cada uma mais barata do que a de um processo completo. Em máquinas com vários núcleos, threads distintas podem executar de fato ao mesmo tempo, o que caracteriza paralelismo.

Um padrão comum em servidores é o pool de threads: em vez de criar uma thread nova a cada pedido recebido, o programa cria um número fixo delas na inicialização e todas ficam disponíveis para atender às requisições que chegarem. É o modelo adotado neste trabalho.

3.3 Exclusão mútua e seção crítica

O compartilhamento de memória entre threads traz de volta o problema da concorrência. Quando duas threads alteram a mesma estrutura sem coordenação, o resultado depende da ordem em que o escalonador as executa — situação conhecida como condição de corrida. O trecho de código que acessa o recurso compartilhado é chamado de seção crítica, e a garantia de que apenas uma thread esteja nela por vez é a exclusão mútua (TANENBAUM; BOS, 2016).

O mutex é o mecanismo mais direto para obter essa garantia. Antes de entrar na seção crítica, a thread o tranca; ao sair, destranca. Se outra thread tentar trancá-lo enquanto ele estiver em uso, ficará bloqueada até que seja liberado.

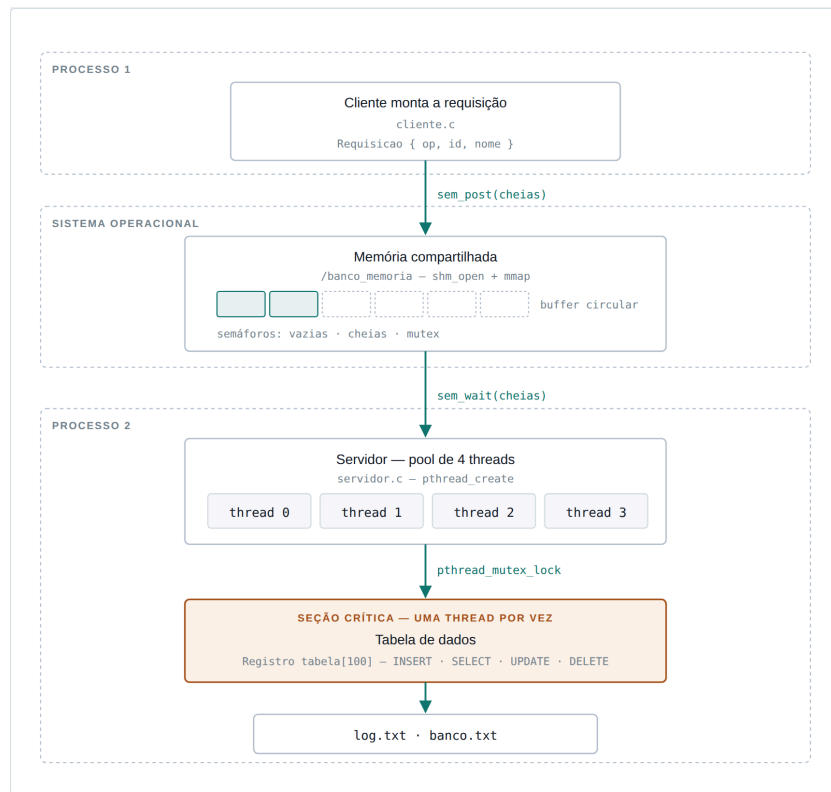
4 DESENVOLVIMENTO

O sistema foi implementado na linguagem C, em ambiente Linux, utilizando as bibliotecas POSIX de memória compartilhada, semáforos e threads. São cinco arquivos, compilados pelo utilitário make em dois executáveis independentes.

4.1 Arquitetura e fluxo do sistema

A Figura 1 apresenta o caminho percorrido por uma requisição desde o cliente até o arquivo de log. Os dois processos não se conhecem diretamente: a única ligação entre eles é a região de memória compartilhada criada pelo sistema operacional.

Figura 1 – Fluxo de uma requisição no sistema



Fonte: os autores (2026).

O percurso é o seguinte. O cliente monta uma estrutura do tipo `Requisicao`, contendo a operação, o identificador e o nome, e a deposita no buffer que fica dentro da memória compartilhada. Uma das threads do servidor retira essa requisição do buffer, tranca o mutex, executa a operação sobre a tabela de dados e destranca. O resultado é então gravado no arquivo de log e exibido na tela. Ao final, quando o cliente avisa que terminou, o servidor grava a tabela no arquivo `banco.txt`.

4.2 Estrutura dos arquivos

O Quadro 1 descreve a função de cada arquivo do projeto.

Quadro 1 – Arquivos do projeto

Arquivo	Função
<code>cliente.c</code>	Monta as requisições e as escreve na memória compartilhada
<code>servidor.c</code>	Cria a memória compartilhada, sobe as quatro threads e executa as operações na tabela
<code>banco.h</code>	Estruturas e constantes que os dois programas precisam conhecer
<code>banco.txt</code>	Registros salvos, no formato <code>id;nome</code>
<code>Makefile</code>	Compila os dois programas
<code>log.txt</code>	Gerado em execução: registra o que cada thread fez

Fonte: os autores (2026).

O arquivo banco.h merece uma observação. Como cliente e servidor são processos separados e compilados em executáveis diferentes, ambos precisam concordar sobre o formato exato das estruturas que atravessam a memória compartilhada. Se um deles alterasse a definição da estrutura Requisicao, o outro passaria a interpretar os mesmos bytes de forma equivocada. Manter essas definições em um cabeçalho comum resolve o problema.

4.3 Estruturas de dados

O Programa 1 mostra as estruturas definidas em banco.h. O registro segue exatamente o formato indicado no enunciado. O buffer é circular: o cliente escreve na posição indicada por fim e o servidor lê na posição indicada por inicio, e ambos os índices voltam ao começo do vetor quando chegam ao final.

Programa 1 – Estruturas compartilhadas pelos dois processos (banco.h)

```
typedef struct {
    int id;
    char nome[50];
} Registro;

typedef enum { INSERT, DELETE, SELECT, UPDATE, FIM } Operacao;

typedef struct {
    Operacao op;
    int id;
    char nome[50];
} Requisicao;

typedef struct {
    Requisicao requisicoes[MAX_REQUISICOES];
    int inicio; /* de onde o servidor le */
    int fim; /* onde o cliente escreve */
} Buffer;
```

Fonte: os autores (2026).

4.4 Comunicação entre os processos

O servidor cria a região de memória compartilhada com a chamada shm_open e a mapeia no seu espaço de endereçamento com mmap. Em seguida cria três semáforos nomeados: vazias, iniciado com o número de posições livres do buffer; cheias, iniciado em zero; e mutex, iniciado em um. O cliente, ao ser executado, abre os mesmos objetos pelo nome.

O Programa 2 mostra o envio de uma requisição. Os dois primeiros semáforos resolvem o problema do produtor e do consumidor: o cliente fica bloqueado quando o buffer está cheio e as threads do servidor ficam bloqueadas quando ele está vazio, sem que nenhum dos lados precise verificar o buffer repetidamente e consumir processador à toa. O terceiro garante que apenas um dos processos altere os índices por vez.

Programa 2 – Envio de uma requisição pela memória compartilhada (cliente.c)

```
sem_wait(sem_vazias);    /* espera ter slot livre          */
sem_wait(sem_mutex);    /* entra na regiao do buffer          */

buffer->requisicoes[buffer->fim] = req;
buffer->fim = (buffer->fim + 1) % MAX_REQUISICOES;

sem_post(sem_mutex);    /* libera o buffer                    */
sem_post(sem_cheias);   /* avisa que chegou requisicao        */
```

Fonte: os autores (2026).

4.5 Pool de threads

Na inicialização, o servidor cria quatro threads com `pthread_create`, todas executando a mesma função. Cada uma recebe apenas o seu número de identificação, usado para marcar as linhas do log.

Programa 3 – Criação do pool de threads (servidor.c)

```
for (i = 0; i < NUM_THREADS; i++) {
    numeros[i] = i;
    pthread_create(&threads[i], NULL, trabalhador, &numeros[i]);
}

for (i = 0; i < NUM_THREADS; i++)
    pthread_join(threads[i], NULL);
```

Fonte: os autores (2026).

A chamada `pthread_join` faz o programa principal esperar que todas as threads terminem antes de salvar a tabela em arquivo. Sem ela, o servidor poderia gravar o banco enquanto alguma thread ainda estivesse alterando a tabela.

4.6 Seção crítica

Cada thread executa um laço: retira uma requisição do buffer, tranca o mutex, executa a operação sobre a tabela e destranca. O trecho entre `pthread_mutex_lock` e `pthread_mutex_unlock` é a seção crítica do programa, exibida no Programa 4.

Programa 4 – Seção crítica protegida por mutex (servidor.c)

```
pthread_mutex_lock(&mutex_tabela);    /* ---- entra ---- */

switch (req.op) {
    case INSERT: op_insert(req.id, req.nome, resultado); break;
    case SELECT: op_select(req.id, resultado);           break;
    case UPDATE: op_update(req.id, req.nome, resultado); break;
    case DELETE: op_delete(req.id, resultado);           break;
}

usleep(300000); /* pausa proposital, para acompanhar na tela */

pthread_mutex_unlock(&mutex_tabela); /* ---- sai ----- */
```

Fonte: os autores (2026).

A chamada `usleep` no meio do trecho não faz parte do algoritmo: ela foi inserida de propósito para que cada operação demore trezentos milissegundos. Sem essa pausa, o sistema atende a todas as requisições em poucos milissegundos e não é possível acompanhar o revezamento das threads durante a apresentação. A pausa também é a razão dos tempos medidos na seção 5.3.

Sem a proteção do mutex, duas threads poderiam executar a operação de inserção simultaneamente: ambas leriam a variável que conta os registros com o mesmo valor, escreveriam na mesma posição do vetor e um dos registros seria perdido. É exatamente a condição de corrida descrita na seção 3.3.

5 RESULTADOS

5.1 Execução do sistema

O sistema foi executado com o banco inicial contendo quatro registros e com o cliente enviando oito requisições, escolhidas de modo a exercitar as quatro operações e também os casos de erro: uma busca por identificador inexistente e uma inserção com identificador repetido. O Quadro 2 reproduz a saída do servidor.

Quadro 2 – Saída do servidor com quatro threads

```

=== SERVIDOR ===
banco.txt carregado com 4 registro(s)
memoria compartilhada e semaforos criados
4 threads criadas, esperando requisicoes...

[thread 0] INSERT id=7 nome='Joao' -> OK
[thread 3] SELECT nome WHERE id=5 -> 'Pedro'
[thread 3] DELETE WHERE id=2 -> OK
[thread 3] SELECT nome WHERE id=2 -> nao encontrado
[thread 3] INSERT id=1 -> ERRO: id ja existe
[thread 2] INSERT id=10 nome='Maria' -> OK
[thread 1] UPDATE id=7 nome='Joao Paulo' -> OK
[thread 0] SELECT nome WHERE id=7 -> 'Joao Paulo'

banco.txt salvo com 5 registro(s)
servidor encerrado

```

Fonte: os autores (2026).

Todas as operações produziram o resultado esperado: o registro de identificador 7 foi inserido e depois alterado, o de identificador 2 foi removido, a busca por ele passou a não encontrar nada, e a tentativa de inserir um identificador já existente foi recusada. O Quadro 3 mostra o conteúdo do arquivo banco.txt antes e depois da execução.

Quadro 3 – Conteúdo do banco antes e depois da execução

Antes	Depois
1;Ana	1;Ana
2;Bruno	3;Carla
3;Carla	5;Pedro
5;Pedro	7;Joao Paulo
—	10;Maria

Fonte: os autores (2026).

5.2 Ordem de atendimento

Uma diferença visível entre executar o servidor com uma única thread e com quatro está na ordem em que as requisições são atendidas, apresentada no Quadro 4. Com uma thread, as operações são processadas exatamente na ordem em que foram enviadas. Com quatro, a ordem muda: as threads retiram requisições do buffer em paralelo e o escalonador decide qual delas avança primeiro.

Quadro 4 – Ordem de atendimento com uma e com quatro threads

#	Com 1 thread	Com 4 threads
1	[thread 0] INSERT id=7	[thread 0] INSERT id=7
2	[thread 0] SELECT id=5	[thread 3] SELECT id=5
3	[thread 0] INSERT id=10	[thread 3] DELETE id=2
4	[thread 0] UPDATE id=7	[thread 3] SELECT id=2
5	[thread 0] SELECT id=7	[thread 3] INSERT id=1
6	[thread 0] DELETE id=2	[thread 2] INSERT id=10
7	[thread 0] SELECT id=2	[thread 1] UPDATE id=7
8	[thread 0] INSERT id=1	[thread 0] SELECT id=7

Fonte: os autores (2026).

Na execução com quatro threads, todas as quatro receberam trabalho: a thread 3 atendeu a cinco requisições, a thread 0 a três, e as threads 1 e 2 a duas cada. Essa distribuição não é fixa e muda a cada execução, porque depende do escalonador do sistema operacional.

5.3 Tempo de execução

Para avaliar o efeito do número de threads sobre o desempenho, o mesmo conjunto de oito requisições foi submetido a quatro configurações do servidor, variando apenas a quantidade de threads do pool. O Quadro 5 apresenta os tempos medidos.

Quadro 5 – Tempo total de execução por número de threads

Threads no pool	Tempo total	Tempo por requisição
1	2,41 s	0,30 s
2	2,41 s	0,30 s
4	2,41 s	0,30 s
8	2,41 s	0,30 s

Fonte: os autores (2026).

O tempo total permaneceu constante em 2,41 segundos em todas as configurações, o que equivale exatamente às oito requisições multiplicadas pelos trezentos milissegundos de processamento de cada uma. Aumentar o número de threads não reduziu o tempo de execução. A explicação está na seção seguinte.

6 ANÁLISE E DISCUSSÃO

O resultado do Quadro 5 pode parecer contraditório em um trabalho sobre paralelismo, mas é justamente o que se espera desta implementação, e ilustra bem o conceito de seção crítica. Todo o processamento de uma requisição — inclusive a pausa de trezentos milissegundos — acontece dentro do trecho protegido pelo mutex. Como o mutex admite apenas uma thread por vez, as operações são executadas uma após a outra, e o tempo total é a soma dos tempos individuais, independentemente de quantas threads existam.

Isso não significa que as threads não estejam funcionando. O Quadro 4 mostra que as quatro receberam requisições e que a ordem de atendimento mudou em relação à execução com uma única thread — sinal de que elas de fato disputaram o trabalho. O que acontece é que a disputa se resolve na porta da seção crítica: enquanto uma thread está lá dentro, as outras aguardam.

Esse comportamento corresponde ao que a literatura descreve como o limite imposto pela parte sequencial de um programa. O ganho de desempenho obtido com paralelismo é proporcional à fração do trabalho que pode ser executada simultaneamente; se essa fração é nula, como no caso presente, não há ganho a obter. Para que o número de threads influenciasse o tempo, seria necessário que parte do processamento ocorresse fora da seção crítica — por exemplo, se a preparação da resposta ou a gravação no log fossem tarefas custosas, elas poderiam ser executadas em paralelo, deixando sob o mutex apenas o acesso efetivo à tabela.

Há ainda um ponto que merece registro. Como as requisições são retiradas do buffer em paralelo, não existe garantia de ordem entre elas. Na execução do Quadro 2, a inserção do registro 7 e a alteração desse mesmo registro foram executadas por threads diferentes e o resultado final ficou correto, mas isso não é assegurado pelo sistema: em outra execução, a alteração poderia ser atendida antes da inserção e retornaria a mensagem de registro não encontrado. Essa é uma consequência direta do paralelismo pedido pelo enunciado. O que o mutex garante não é a ordem, e sim a consistência: a tabela nunca é alterada por duas threads ao mesmo tempo e, portanto, nunca fica em estado inválido.

Quanto ao mecanismo de comunicação, a escolha da memória compartilhada mostrou-se adequada ao problema. Por ser a forma mais rápida de IPC — os dados não passam pelo núcleo do sistema a cada troca, como ocorre com pipes —, ela exige em contrapartida que a sincronização seja feita explicitamente pelo programador, o que foi resolvido com os três semáforos nomeados. Durante os

testes, executar o cliente sem o servidor no ar produz uma mensagem de erro clara, já que a região de memória ainda não existe.

7 CONSIDERAÇÕES FINAIS

O sistema implementado atende aos requisitos do enunciado: cliente e servidor são executáveis distintos, executados como processos separados; a comunicação entre eles é feita por memória compartilhada POSIX, coordenada por semáforos nomeados; o servidor processa as requisições com um pool de quatro threads; e o acesso à tabela compartilhada é protegido por um mutex. As quatro operações previstas foram implementadas e verificadas, incluindo os casos de erro, e os resultados são gravados em arquivo de log.

Do ponto de vista do aprendizado, o trabalho tornou concretos três conceitos que, estudados isoladamente, permanecem abstratos. A memória compartilhada mostrou por que dois processos não conseguem simplesmente enxergar as variáveis um do outro. Os semáforos mostraram a diferença entre esperar bloqueado e verificar repetidamente uma condição. E o mutex mostrou, nos tempos medidos, que proteger uma seção crítica tem um custo: a serialização do acesso ao recurso compartilhado é o preço da consistência dos dados.

Como continuidade, seria interessante reduzir o trecho protegido pelo mutex ao mínimo necessário, deixando fora dele as tarefas que não tocam a tabela, e comparar os tempos obtidos com os do Quadro 5. Outra extensão natural seria substituir o mutex por um mecanismo que permita várias leituras simultâneas, já que operações de consulta não alteram a tabela e, portanto, não precisariam ser serializadas entre si.

REFERÊNCIAS

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Fundamentos de sistemas operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

STEVENS, W. Richard; RAGO, Stephen A. Advanced programming in the UNIX environment. 3. ed. Boston: Addison-Wesley, 2013.

TANENBAUM, Andrew S.; BOS, Herbert. Sistemas operacionais modernos. 4. ed. São Paulo: Pearson Education do Brasil, 2016.

THE OPEN GROUP. The Open Group Base Specifications Issue 7, 2018 edition: IEEE Std 1003.1-2017. Disponível em: <https://pubs.opengroup.org/onlinepubs/9699919799/>. Acesso em: 21 set. 2026.